



ServerPack Cloud Tenant Virtual Machine Allocator

CASE STUDY - 45

Name: Manan Jain

Roll Number: 150096725166

Institution: ITM Skills University

Subject: React Development

Batch: 2025-29

GitHub: <https://github.com/mananjain20/virtual-machine-allocator-dsa>

Problem Statement

ServerPack is the management platform for a massive data center that assigns virtual machines to customers, monitors hardware utilization, and optimizes power consumption. The existing system has several critical problems:

- Looking up physical server resource stats is inconsistent and slow
- Incorrect VM resizing operations cannot be rolled back
- Provisioning requests arrive in bulk but get handled randomly
- The system cannot rank servers by current utilization — some machines run at full capacity while others are nearly idle
- No way to consolidate workloads during off-peak hours to save power

Objectives

1. Maintain an organized hardware inventory with fast $O(1)$ server lookups
2. Support rollback of incorrect VM resize operations using a stack
3. Process VM provisioning requests in strict FIFO order
4. Verify VM operating systems against a sorted valid OS list
5. Rank physical servers by utilization for load balancing
6. Model datacenter network connectivity as a weighted graph
7. Calculate minimum latency path between two servers using Dijkstra's algorithm
8. Consolidate workloads onto the fewest servers to maximize power savings

Design / Architecture

The system is built around a single core class CloudManager that manages all operations. The design follows a modular approach where each feature is handled by an appropriate data structure.

Core Components

- **CloudManager** — Central controller class managing all 8 system features
- **Server** — Represents a physical machine with CPU, RAM and list of assigned VMs
- **VM** — Represents a virtual machine with ID, operating system and RAM
- **ResizeHistory** — Stores the previous RAM value for undo/rollback operations

Key Modules

- **Inventory Module** — Add and view physical servers
- **Provisioning Module** — Queue and assign VMs to servers in FIFO order
- **Resize Module** — Resize VMs with full undo support
- **Verification Module** — Check OS against a valid sorted list
- **Network Module** — Map server connections and find the fastest path
- **Optimization Module** — Sort servers by load and recommend power savings

Algorithm Description

1. Hardware Inventory — `unordered_map`

Servers are stored in an `unordered_map` keyed by server ID. This provides $O(1)$ average-case access for add and lookup operations. This mirrors how VMware vCenter maintains a hash-indexed host table.

2. Resize Undo — `Stack`

Every resize operation pushes the previous RAM value onto a stack. Calling `undo` pops the top entry and restores the VM to its original configuration. LIFO behavior ensures the most recent resize is always undone first.

3. Provisioning Queue — `Queue`

VM requests are pushed into a queue and processed in strict FIFO order. This ensures fairness across all provisioning requests, similar to RabbitMQ task queues used in OpenStack Nova.

4. OS Verification — `Binary Search`

A sorted vector of valid operating systems is searched using `binary_search`, giving $O(\log n)$

verification time. The list is sorted once and reused for all verification calls.

5. Density Sorter — `std::sort`

Servers are copied into a temporary vector and sorted in descending order by VM count using `std::sort` with a lambda comparator. This directly supports load balancing and utilization Visibility.

6. Datacenter Map — Adjacency List

The network is modeled as an undirected weighted graph using an adjacency list. Each server is a node and each physical connection is a weighted edge representing latency in Milliseconds.

7. Fastest Connection — Dijkstra's Algorithm

A min-heap based priority queue implements Dijkstra's algorithm to find the minimum latency path between any two servers. This is the same approach used by SDN controllers for optimal Routing.

8. Power Saver — Greedy Sort

Servers are sorted by utilization. Underloaded servers (1 or fewer VMs) are flagged with a

recommendation to migrate workloads to the most loaded server, enabling those underloaded servers to be powered off.

Complexity Analysis

Complexity Analysis

1. Add / Lookup Server — Time: $O(1)$ average — Space: $O(n)$
2. Add / Lookup VM — Time: $O(1)$ average — Space: $O(m)$
3. Enqueue VM Request — Time: $O(1)$ — Space: $O(k)$
4. Process VM Request — Time: $O(n)$ — Space: $O(1)$
5. Resize VM — Time: $O(1)$ — Space: $O(r)$
6. Undo Resize — Time: $O(1)$ — Space: $O(1)$
7. OS Verification — Time: $O(\log n)$ — Space: $O(s)$
8. Density Sorter — Time: $O(n \log n)$ — Space: $O(n)$
9. Add Graph Connection — Time: $O(1)$ — Space: $O(E)$
10. Shortest Path Dijkstra — Time: $O((V+E) \log V)$ — Space: $O(V+E)$
11. Power Saver — Time: $O(n \log n)$ — Space: $O(n)$

Note: n = servers, m = VMs, k = queue size, r = resize history, s = OS list size, V = vertices, E = edges

Screenshots of Execution

```
1 #include <iostream>
2 #include <unordered_map>
3 #include <vector>
4 #include <queue>
5 #include <stack>
6 #include <algorithm>
7 #include <limits>
8 using namespace std;
9 // Simple VM record with id, OS and RAM
10 class VM
11 {
12 public:
13     string vmId;
14     string os;
15     int ram;
16
17     VM() {}
18     VM(string id, string o, int r) : vmId(id), os(o), ram(r) {}
19 };
20
21 // Simple server record holding resources and hosted VMs
22 class Server
23 {
24 public:
25     string serverId;
26     int cpu;
27     int ram;
28     vector<string> vmList;
29
30     Server() {}
31     Server(string id, int c, int r) : serverId(id), cpu(c), ram(r) {}
32 };
33
34 // History entry to allow undoing a VM resize
35 struct ResizeHistory
36 {
37     string vmId;
38     int oldRam;
39 };
40
41 // Cloud manager that keeps servers, VMs, network and operations
42 class CloudManager
43 {
44 private:
45     // Map serverId -> Server for fast lookup
46     unordered_map<string, Server> servers;
47     // Map vmId -> VM for fast lookup
48     unordered_map<string, VM> vms;
49
50     // Queue for incoming VM requests (FIFO)
51     queue<VM> requestQueue;
52     // Stack to record resize operations for undo (LIFO)
53     stack<ResizeHistory> resizeStack;
54
55     // Allowed OS names (kept in a vector for easy sorting/search)
56     vector<string> validOS = {"UBUNTU", "WINDOWS", "LINUX", "DEBIAN", "FEDORA", "MACOS"};
57
58     // Network graph as adjacency list: server -> list of (neighbor, latency)
59     unordered_map<string, vector<pair<string, int>>> graph;
60
61 public:
62     // Add a new server with user-provided specs
63     void addServer()
64     {
65         string id;
66         int cpu, ram;
67
68         cout << "Server ID: ";
69         cin >> id;
70         cout << "CPU Cores: ";
71         cin >> cpu;
72         cout << "RAM (GB): ";
73         cin >> ram;
74
75         servers[id] = Server(id, cpu, ram);
76         cout << "Server added.\n";
77     }
78
79     // Print all servers and their basic info
80     void viewServers()
81     {
82         if (servers.empty())
83         {
84             cout << "No servers found.\n";
85             return;
86         }
87
88         for (auto &entry : servers)
89         {
90             Server &s = entry.second;
91             cout << "Server: " << s.serverId
92                  << "\nCPU: " << s.cpu
93                  << "\nRAM: " << s.ram
94                  << "\nVM Count: " << s.vmList.size()
95                  << "\n-----\n";
96         }
97     }
98
99     // Queue a VM creation request from user input
100     void addVMRequest()
101     {
102         string id, os;
103         int ram;
```

```

100 // Handle a VM creation request from user input
101 void addVMRequest()
102 {
103     string id, os;
104     int ram;
105
106     cout << "VM ID: ";
107     cin >> id;
108     cout << "OS: ";
109     cin >> os;
110     cout << "RAM (GB): ";
111     cin >> ram;
112
113     requestQueue.push(VM(id, os, ram));
114     cout << "Request added to queue.\n";
115 }
116
117 // Assign next queued VM to the least-loaded server
118 void processVMRequest()
119 {
120     if (requestQueue.empty())
121     {
122         cout << "No pending requests.\n";
123         return;
124     }
125
126     if (servers.empty())
127     {
128         cout << "Add a server first.\n";
129         return;
130     }
131
132     VM vm = requestQueue.front();
133     requestQueue.pop();
134     // Find server with minimum VM count
135     auto bestServer = servers.begin();
136
137     for (auto it = servers.begin(); it != servers.end(); it++)
138     {
139         if (it->second.vmlist.size() <
140             bestServer->second.vmlist.size())
141         {
142             bestServer = it;
143         }
144     }
145
146     bestServer->second.vmlist.push_back(vm.vmlid);
147
148     vms[vm.vmlid] = vm;
149
150     cout << "VM " << vm.vmlid
151         << " assigned to Server "
152         << bestServer->second.serverId
153         << "\n";
154 }
155
156 // List all VMs with their properties
157 void viewVMs()
158 {
159     if (vms.empty())
160     {
161         cout << "No VMs available.\n";
162         return;
163     }
164
165     for (auto &entry : vms)
166     {
167         VM &vm = entry.second;
168
169         cout << "VM ID: " << vm.vmlid
170             << "\nOS: " << vm.os
171             << "\nRAM: " << vm.ram << " GB"
172             << "\n-----\n";
173     }
174 }
175
176 // Change RAM of a VM and record history for undo
177 void resizeVM()
178 {
179     string id;
180     int newRam;
181
182     cout << "VM ID: ";
183     cin >> id;
184
185     if (vms.find(id) == vms.end())
186     {
187         cout << "VM not found.\n";
188         return;
189     }
190
191     cout << "New RAM: ";
192     cin >> newRam;
193
194     resizeStack.push({id, vms[id].ram});
195     vms[id].ram = newRam;
196
197     cout << "VM resized.\n";
198 }

```

```

192     vms[id].ram = newRam;
193
194     cout << "VM resized.\n";
195 }
196
197 // Revert the last VM resize using the resize stack
198 void undoResize()
199 {
200     if (resizeStack.empty())
201     {
202         cout << "Nothing to undo.\n";
203         return;
204     }
205
206     ResizeHistory last = resizeStack.top();
207     resizeStack.pop();
208
209     vms[last.vmlId].ram = last.oldRam;
210
211     cout << "Undo successful.\n";
212 }
213
214 // Verify whether an OS is in the allowed list
215 void systemCheck()
216 {
217     // sort OS list once so we can binary search it quickly
218     sort(validOS.begin(), validOS.end());
219
220     string os;
221     cout << "Enter OS to verify: ";
222     cin >> os;
223
224     if (binary_search(validOS.begin(), validOS.end(), os))
225         cout << "OS verified.\n";
226     else
227         cout << "OS not found.\n";
228 }
229
230 // Show servers ordered by number of VMs (high -> low)
231 void densitySorter()
232 {
233     vector<Server> list;
234
235     for (auto &entry : servers)
236         list.push_back(entry.second);
237
238     // sort servers by VM count (descending) to show utilization
239     sort(list.begin(), list.end(),
240          [](Server a, Server b)
241          {
242              return a.vmlList.size() > b.vmlList.size();
243          });
244
245     cout << "\nServers by utilization:\n";
246
247     for (auto &s : list)
248     {
249         cout << s.serverId
250              << " -> " << s.vmlList.size()
251              << " VMs\n";
252     }
253 }
254
255 // Add a bidirectional network link between two servers
256 void addConnection()
257 {
258     string a, b;
259     int latency;
260
261     cout << "Server A: ";
262     cin >> a;
263     cout << "Server B: ";
264     cin >> b;
265     cout << "Latency: ";
266     cin >> latency;
267
268     graph[a].push_back({b, latency});
269     graph[b].push_back({a, latency});
270
271     cout << "Connection added.\n";
272 }
273
274 // Print adjacency list of the datacenter network
275 void showMap()
276 {
277     for (auto &entry : graph)
278     {
279         cout << "\n"
280              << entry.first << " -> ";
281
282         for (auto &edge : entry.second)
283         {
284             cout << "(" << edge.first
285                  << ", " << edge.second
286                  << "ms) ";
287         }
288         cout << "\n";
289     }
290
291     // Connect minimum latency with between two servers (Bridges)

```

```

213 // (Returns minimum latency path between two servers - Dijkstra)
214 void fastestConnection()
215 {
216     string source, destination;
217
218     cout << "Source Server: ";
219     cin >> source;
220
221     cout << "Destination Server: ";
222     cin >> destination;
223
224     unordered_map<string, int> dist;
225
226     for (auto &node : graph)
227         dist[node.first] = INT_MAX;
228
229     if (graph.find(source) == graph.end())
230     {
231         cout << "Source not found.\n";
232         return;
233     }
234
235     priority_queue<pair<int, string>,
236                 vector<pair<int, string>>,
237                 greater<pair<int, string>>>
238     > pq;
239
240     dist[source] = 0;
241     pq.push({0, source});
242
243     // Dijkstra main loop: relax edges until all shortest paths found
244     while (!pq.empty())
245     {
246         auto current = pq.top();
247         pq.pop();
248
249         int currentDist = current.first;
250         string currentNode = current.second;
251
252         for (auto &neighbor : graph[currentNode])
253         {
254             string nextNode = neighbor.first;
255             int weight = neighbor.second;
256
257             if (currentDist + weight < dist[nextNode])
258             {
259                 dist[nextNode] = currentDist + weight;
260                 pq.push({dist[nextNode], nextNode});
261             }
262         }
263     }
264
265     if (dist[destination] == INT_MAX)
266         cout << "No path found.\n";
267     else
268         cout << "Minimum latency = "
269             << dist[destination]
270             << " ms\n";
271
272     // Recommend servers to power off by finding lightly used ones
273     void powerDown()
274     {
275         vector<Server> list;
276
277         for (auto &entry : servers)
278             list.push_back(entry.second);
279
280         // sort servers by VM count (descending) to find light ones
281         sort(list.begin(), list.end(),
282              [](Server &a, Server &b)
283              {
284                  return a.vmlist.size() > b.vmlist.size();
285              });
286
287         cout << "LowPower Server Recommendation\n";
288
289         for (auto &s : list)
290         {
291             if (s.vmlist.size() <= 1)
292             {
293                 cout << "Server " << s.serverId
294                     << " (- << s.vmlist.size() << " VM) - move to Server "
295                     << list[0].serverId
296                     << " | Power off " << s.serverId << " to save power.\n";
297             }
298         }
299     }
300 }
301
302 // Menu-driven main to interact with the CloudManager
303 int main()
304 {
305     CloudManager manager;
306     int choice;
307
308     do
309     {
310         cout << "***** SERVERSPACE CLOUD SYSTEM *****\n";
311         cout << "1. Add Server\n";
312         cout << "2. View Servers\n";
313     }

```

```

381 int choice;
382
383 do
384 {
385     cout << "\n===== SERVERPACK CLOUD SYSTEM =====\n";
386     cout << "1. Add Server\n";
387     cout << "2. View Servers\n";
388     cout << "3. Add VM Request\n";
389     cout << "4. Process VM Request\n";
390     cout << "5. View VMs\n";
391     cout << "6. Resize VM\n";
392     cout << "7. Undo Resize\n";
393     cout << "8. System Check\n";
394     cout << "9. Density Sorter\n";
395     cout << "10. Add Connection\n";
396     cout << "11. Show Datacenter Map\n";
397     cout << "12. Fastest Connection\n";
398     cout << "13. Power Saver\n";
399     cout << "0. Exit\n";
400     cout << "Choice: ";
401     cin >> choice;
402     switch (choice)
403     {
404     case 1:
405         manager.addServer();
406         break;
407     case 2:
408         manager.viewServers();
409         break;
410     case 3:
411         manager.addVMRequest();
412         break;
413     case 4:
414         manager.processVMRequest();
415         break;
416     case 5:
417         manager.viewVMs();
418         break;
419     case 6:
420         manager.resizeVM();
421         break;
422     case 7:
423         manager.undoResize();
424         break;
425     case 8:
426         manager.systemCheck();
427         break;
428     case 9:
429         manager.densitySorter();
430         break;
431     case 10:
432         manager.addConnection();
433         break;
434     case 11:
435         manager.showMap();
436         break;
437     case 12:
438         manager.fastestConnection();
439         break;
440     case 13:
441         manager.powerSaver();
442         break;
443     case 0:
444         cout << "Thank You!\n";
445         break;
446     default:
447         cout << "Invalid Choice.\n";
448     }
449 } while (choice != 0);
450 return 0;
451 }
452

```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
mananjain@Manans-MacBook-Air:DSA(Final) % cd "/Users/mananjain/Documents/DSA(Final)/" && g++ main.cpp -o main && "/Users/mananjain/Documents/DSA(Final)/main"
Choices: 2

Servers: 53
CPU: 32
RAM: 16
VM Count: 2

Servers: 52
CPU: 32
RAM: 8
VM Count: 1

Servers: 51
CPU: 32
RAM: 8
VM Count: 1

===== SERVERPACK CLOUD SYSTEM =====
1. Add Server
2. View Servers
3. Add VM Request
4. Process VM Request
5. View VMs
6. Resize VM
7. Undo Resize
8. System Check
9. Density Sorter
10. Add Connection
11. Show Datacenter Map
12. Fastest Connection
13. Power Saver
0. Exit
Choices: 5

VM ID: V4
OS: LINUX
RAM: 8 GB

VM ID: V2
OS: WINDOWS
RAM: 8 GB

VM ID: V3
OS: UBUNTU
RAM: 4 GB

VM ID: V1
OS: MACOS
RAM: 16 GB

===== SERVERPACK CLOUD SYSTEM =====
1. Add Server
2. View Servers
3. Add VM Request
4. Process VM Request
5. View VMs
6. Resize VM
7. Undo Resize
8. System Check
9. Density Sorter
10. Add Connection
11. Show Datacenter Map
12. Fastest Connection
13. Power Saver
0. Exit
Choices: 6
```

```
mananjain@Manans-MacBook-Air:DSA(Final) % cd "/Users/mananjain/Documents/DSA(Final)/" && g++ main.cpp -o main && "/Users/mananjain/Documents/DSA(Final)/main"
===== SERVERPACK CLOUD SYSTEM =====
1. Add Server
2. View Servers
3. Add VM Request
4. Process VM Request
5. View VMs
6. Resize VM
7. Undo Resize
8. System Check
9. Density Sorter
10. Add Connection
11. Show Datacenter Map
12. Fastest Connection
13. Power Saver
0. Exit
Choices: 6
VM ID: V1
New RAM: 8
VM resized.

===== SERVERPACK CLOUD SYSTEM =====
1. Add Server
2. View Servers
3. Add VM Request
4. Process VM Request
5. View VMs
6. Resize VM
7. Undo Resize
8. System Check
9. Density Sorter
10. Add Connection
11. Show Datacenter Map
12. Fastest Connection
13. Power Saver
0. Exit
Choice: 7
Undo successful.

===== SERVERPACK CLOUD SYSTEM =====
1. Add Server
2. View Servers
3. Add VM Request
4. Process VM Request
5. View VMs
6. Resize VM
7. Undo Resize
8. System Check
9. Density Sorter
10. Add Connection
11. Show Datacenter Map
12. Fastest Connection
13. Power Saver
0. Exit
Choice: 8
Enter OS to verify: LINUX
OS verified.

===== SERVERPACK CLOUD SYSTEM =====
1. Add Server
2. View Servers
3. Add VM Request
4. Process VM Request
5. View VMs
6. Resize VM
7. Undo Resize
8. System Check
9. Density Sorter
10. Add Connection
11. Show Datacenter Map
12. Fastest Connection
13. Power Saver
0. Exit
Choices: 1
```

```
0. Exit
Choice: 9

Servers by utilization:
S3 -> 2 VMs
S2 -> 1 VMs
S1 -> 1 VMs

===== SERVERPACK CLOUD SYSTEM =====
1. Add Server
2. View Servers
3. Add VM Request
4. Process VM Request
5. View VMs
6. Resize VM
7. Undo Resize
8. System Check
9. Density Sorter
10. Add Connection
11. Show Datacenter Map
12. Fastest Connection
13. Power Saver
0. Exit
Choice: 10
Server A: S1
Server B: S2
Latency: 2
Connection added.

===== SERVERPACK CLOUD SYSTEM =====
1. Add Server
2. View Servers
3. Add VM Request
4. Process VM Request
5. View VMs
6. Resize VM
7. Undo Resize
8. System Check
9. Density Sorter
10. Add Connection
11. Show Datacenter Map
12. Fastest Connection
13. Power Saver
0. Exit
Choice: 10
Server A: S2
Server B: S3
Latency: 1
Connection added.

===== SERVERPACK CLOUD SYSTEM =====
1. Add Server
2. View Servers
3. Add VM Request
4. Process VM Request
5. View VMs
6. Resize VM
7. Undo Resize
8. System Check
9. Density Sorter
10. Add Connection
11. Show Datacenter Map
12. Fastest Connection
13. Power Saver
0. Exit
Choice: 10
Server A: S1
Server B: S3
Latency: 3
Connection added.
```

```

Choice: 11
S3 -> (S2, 1ms) (S1, 3ms)
S2 -> (S1, 2ms) (S3, 1ms)
S1 -> (S2, 2ms) (S3, 3ms)

===== SERVERPACK CLOUD SYSTEM =====
1. Add Server
2. View Servers
3. Add VM Request
4. Process VM Request
5. View VMs
6. Resize VM
7. Undo Resize
8. System Check
9. Density Sorter
10. Add Connection
11. Show Datacenter Map
12. Fastest Connection
13. Power Saver
0. Exit
Choice: 12
Source Server: S1
Destination Server: S3
Minimum latency = 3 ms

===== SERVERPACK CLOUD SYSTEM =====
1. Add Server
2. View Servers
3. Add VM Request
4. Process VM Request
5. View VMs
6. Resize VM
7. Undo Resize
8. System Check
9. Density Sorter
10. Add Connection
11. Show Datacenter Map
12. Fastest Connection
13. Power Saver
0. Exit
Choice: 13

Power Saver Recommendation
Server S2 (1 VM) -> move to Server S3 | Power off S2 to save power.
Server S1 (1 VM) -> move to Server S3 | Power off S1 to save power.

===== SERVERPACK CLOUD SYSTEM =====
1. Add Server
2. View Servers
3. Add VM Request
4. Process VM Request
5. View VMs
6. Resize VM
7. Undo Resize
8. System Check
9. Density Sorter
10. Add Connection
11. Show Datacenter Map
12. Fastest Connection
13. Power Saver
0. Exit

```

Results and Findings

- All 8 required features were successfully implemented and tested
- `unordered_map` provided consistent $O(1)$ server and VM lookups — significantly faster than linear search
- Stack-based undo proved reliable for both single and multiple sequential resize rollbacks

- The provisioning queue maintained strict FIFO order across all test cases
- Binary search on the sorted OS list correctly verified valid and invalid operating systems
- Dijkstra's algorithm correctly computed minimum latency paths across multi-hop server connections
- The density sorter successfully ranked servers from most to least utilized
- The power saver correctly identified underloaded servers and recommended consolidation targets
- The system architecture mirrors real-world platforms — VMware vCenter (inventory), OpenStack Nova (queue), OpenStack Neutron (network graph)

Conclusion

The ServerPack Cloud Tenant Virtual Machine Allocator successfully demonstrates the practical application of core data structures — hash maps, stacks, queues, graphs, and heaps — in solving real-world datacenter management challenges.

Each data structure was chosen deliberately based on its time and space efficiency for the specific use case it serves. The system is modular, readable, and extensible, and reflects the architectural patterns found in industry-grade platforms like VMware and OpenStack.

GitHub Repository

The complete source code, README, and screenshots are available at:

[**https://github.com/mananjain20/virtual-machine-allocator-dsa**](https://github.com/mananjain20/virtual-machine-allocator-dsa)